

# RAI: Safe Pipelined Epoch Transitions for Account-Local Payments

Anonymous Author(s)

## Abstract

Single-writer payments finalize from account-local quorums without a global transaction sequence. Replacing the validators that issue those quorums is the hard part: votes already released by the outgoing committee can still form a final certificate after the handoff, and a successor that does not honor them may finalize a conflicting history. RAI is an epoch-transition protocol that lets the next epoch’s account voting proceed while the previous epoch closes, and that keeps the two safe against each other. Each outgoing validator freezes one signed report committing to a tagged ledger set and to the blocks it voted for outside that set. RAI defines which report selections are eligible checkpoint inputs and a deterministic construction that turns them into a checkpoint carrying finality and locks. Two components are left abstract: a *convergence* service that makes frozen reports reconstructible at every correct validator, and a *consensus* service that decides one eligible checkpoint per epoch. RAI is safe and terminates given the safety and termination of those two services, together with explicit fault, retained-data, and progress assumptions. Lagged committees, a predecessor gate for new work, and a narrow overlap exception make the pipeline safe without stopping account voting; the outgoing committee keeps notarizing new work, without finality, until it installs the checkpoint, so that the exception covers blocks published during the close. We instantiate convergence with rateless set reconciliation and consensus with a Kudzu-style election in RsNano. The prototype keeps median non-fork latency near 100 ms and, without forks, p95 latency below 0.31 s across three checkpoints at an offered 2,000 blocks/s, and it settles forked workloads, with one offline or one Byzantine validator, that the unmodified node does not finish; under forks, blocks whose late notarization splits still wait for the checkpoint.

## 1 Introduction

Single-writer value transfer does not require every payment to occupy a position in a global transaction sequence. An account owner orders its own updates; validators only have to prevent incompatible owner-signed histories from becoming final. The consensus-number characterization of asset transfer formalizes this distinction, and payment systems already exploit it [1, 5, 10]. The problem addressed here is what happens when the validators issuing those account-local votes are replaced.

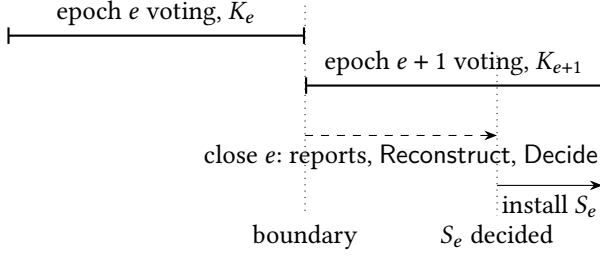
A reconfiguration must decide which old outcomes the successor is obliged to preserve. Consider a payment for

which enough outgoing validators have released signatures to form a final certificate, while network delay keeps those signatures apart, so that no checkpoint builder sees the certificate. If the checkpoint omits the payment and lets an incompatible successor history through, the delayed signatures later establish a valid old-epoch final outcome. The checkpoint was agreed, but it was not safe. We call such an outcome *latent-final*, and we extend the obligation to certificates that outgoing validators which have not yet stopped may still complete.

The obvious remedy is to stop: end account voting, wait until every certificate has settled, then agree on a snapshot. That costs a pause at every epoch boundary, and the pause lasts as long as the slowest certificate. RAI avoids it. It is a protocol for *pipelined* epoch transitions: epoch  $e + 1$  account voting starts at each validator’s local boundary, while epoch  $e$  is still closing, and the two are kept safe against each other by rules that are local to the account path. The closing side freezes one signed *report* per outgoing validator, committing to a tagged view of its ledger and to the blocks it voted for outside that view. A deterministic construction turns a selection of reports into a checkpoint that preserves finality and carries obligation locks. The successor committee, known two epochs ahead, verifies and installs the decided checkpoint without voting on it.

RAI deliberately leaves two things abstract. Reports commit to sets by their roots; a *convergence* service makes every correct reporter’s frozen sets reconstructible at every correct validator. A *consensus* service decides one checkpoint per epoch among the candidates RAI declares eligible. RAI specifies the interfaces and the properties it needs from both, defines the report format and the eligibility predicate that the consensus instance must enforce, and proves safety and termination assuming the safety and termination of the two services. Any set-reconciliation mechanism and any Byzantine agreement protocol with external validity can fill the two slots; our prototype uses rateless set reconciliation and a Kudzu-style election, and Section 9 describes only that instantiation.

**Contributions.** First, RAI specifies pipelined epoch transitions for account-local finality: lagged committees, local boundaries, a predecessor gate for fresh successor work, a narrow prefix-wide overlap exception that lets settlement continue before the predecessor checkpoint is known, and late notarization, by which the outgoing committee extends that exception to blocks published during the close. Second, it defines immutable signed reports and a report-exposure



**Figure 1.** A pipelined transition. Epoch  $e + 1$  account voting starts at the boundary; the close of  $e$  overlaps it; ordinary  $e + 1$  work finalizes once  $S_e$  is installed, and the narrow overlap exception of Section 4.3 finalizes some of it earlier.

invariant that covers latent and late finality, and a deterministic checkpoint construction over eligible report selections. Third, it states the convergence and consensus interfaces as explicit assumptions and proves transfer safety and conditional termination relative to them. Fourth, it reports a single-host prototype in RsNano whose nine workload variants, with forks and with one offline or one Byzantine validator, settle to identical state on every validator while the unmodified node finishes none of the forked ones, and it measures what the pipeline costs in latency tails with and without late notarization.

**Scope of the claims.** The results are protocol arguments, not machine-checked proofs. The main protocol uses a conservative overlap rule that covers every unresolved position being finalized; a fresh child does not inherit cross-epoch safety merely because its parent was notarized twice. Section 9 reports single runs on one host; they illustrate behavior under load and do not validate the proofs’ assumptions. The single-writer restriction and the two instantiated services are not claimed as contributions.

## 2 Overview

### 2.1 The pipeline

Figure 1 shows one epoch transition. Account voting for epoch  $e$  is done by committee  $K_e$ . At its local boundary a validator stops signing epoch- $e$  account votes, freezes its report, and opens epoch  $e + 1$  for the same accounts. The checkpoint  $S_e$  is decided by the outgoing committee from  $N - f$  reports, while epoch  $e + 1$  voting is already under way. The successor committee verifies and installs  $S_e$ ; only then does ordinary epoch- $(e + 1)$  work become final. Committees are lagged by two epochs:  $S_e$  selects the committee that votes in epoch  $e + 2$ , so the committee of  $e + 1$  is known throughout the transition and can prepare without waiting.

Three things must hold for this pipeline to be safe. Old-epoch finality that can still be formed must survive in  $S_e$ : the report rules and the construction of Section 5 guarantee it. New-epoch work must not finalize anything the old

epoch could still contradict: the predecessor gate holds ordinary successor work provisional until  $S_e$  is known, and the overlap exception finalizes early only where old-domain exclusion is proven at every position. The outgoing committee supplies that proof for fresh blocks by *late notarization*: until it installs  $S_e$ , each member also notarizes in epoch  $e$  the blocks it first-votes in epoch  $e + 1$ , with votes that count toward no finality. And every correct validator must install the same  $S_e$ : this is where the consensus service enters, and it decides only among candidates that RAI’s eligibility predicate accepts.

### 2.2 A six-validator example

Let  $N = 6$ ,  $f = p = 1$ , so notarization and normal finality require four votes, fast finality requires five first votes, and recovery requires three reporter first votes. Validators 1, ..., 5 are correct and validator 6 is Byzantine. Validators 1, 2, 3, 4, 6 first-vote for an eligible payment block  $B$ . Vote dissemination is delayed, so no correct reporter has assembled a notarization or final certificate when it freezes its report.

Suppose a checkpoint candidate selects the reports of 1, 2, 3, 5, 6. Each of reporters 1, 2, 3 commits  $h(B)$  in its voted-but-not-ledger set  $G_i$ . Validator 6 may omit its vote; validator 5 did not vote for  $B$ . The three correct memberships meet the recovery threshold. The construction retains  $B$  and its ancestry as a recovery-protected branch without changing balances. Later the five first votes form a fast final certificate, which finalizes the retained branch; a subsequent checkpoint exposes that finality explicitly. The example generalizes in two ways. Every selected report quorum intersects a normal final-vote quorum in at least  $p + 1$  correct signers, each of whom recorded notarization before final-voting. Every selected report quorum intersects a fast first-vote quorum in at least  $f + p + 1$  correct signers, whose obligations appear in their ledger set or in  $G_i$ . These are the two exposure cases of the closure theorem.

### 2.3 What is abstract and what is not

RAI fixes the report format, the tagged ledger projection, the eligibility of a report selection, the deterministic construction, the overlap and installation rules, and the lifecycle of epochs. It does not fix how a validator obtains another reporter’s frozen sets, nor how the outgoing committee agrees on one eligible candidate. Section 3.5 states what RAI needs from these two services; Section 9 describes the instantiation we measured.

## 3 Model, Assumptions, and Abstract Services

### 3.1 Participants, faults, and retained evidence

Clients own accounts and sign blocks. An authenticated committee contains  $N = 3f + 2p + 1$  equal-weight validator identities, where  $f, p \geq 0$  are integers. At most  $f$

**Table 1.** Account and closure thresholds. The example has  $f = p = 1$  and  $N = 6$ .

| Evidence           | Threshold           | Example |
|--------------------|---------------------|---------|
| Notarization NC    | $q = N - f - p$     | 4       |
| Normal finality FC | $q$ final votes     | 4       |
| Fast finality FF   | $N - p$ first votes | 5       |
| Recovery support   | $r = f + p + 1$     | 3       |
| Selected reports   | $N - f$             | 5       |

members of each committee are Byzantine. Corruption is globally static: adversarial identities are fixed throughout the execution, and a correct departing identity is not later compromised. Membership admission, Sybil resistance, and preservation of the per-committee fault bound are deployment assumptions; deterministic committee selection does not establish them.

Signatures are unforgeable and hashes collision resistant. Canonical encodings bind the network session and message domain. An account vote binds its epoch, committee, account, slot, vote type, block hash, and admission-witness digest; the witness identifies the anchor and evidence under which the vote was admitted. Checkpoint votes bind the handoff context and deciding committee. Distinct signatures by one identity never contribute twice to a quorum for the same block.

Correct validators persist their first vote and terminal state before releasing signatures. They retain blocks, signed votes, and finite dependency evidence, and relay them after local voting ends. A certificate is a deterministic local view of sufficient individually verified votes, not a separately gossiped object. A retained item held by a correct validator eventually reaches every correct validator that needs it (the *data service*). Departing validators continue service until correct successors hold durable copies. Epoch labels alone do not protect against stolen old keys; adaptive or mobile corruption is outside the model.

### 3.2 Quorums and the role of $p$

The account path uses

$$N = 3f + 2p + 1, \quad q = 2f + p + 1, \quad r = f + p + 1. \quad (1)$$

Table 1 distinguishes first, final, and report quorums. The parameter  $p$  is the number of first-vote responses the fast path can omit, not an additional fault allowance; increasing  $p$  at fixed  $f$  increases committee size. We repeatedly use

$$2q - N = f + 1, \quad (2)$$

$$q - 2f = p + 1, \quad (3)$$

$$(N - p) - 2f = f + p + 1 = r. \quad (4)$$

### 3.3 Accounts and canonical state

Each account is a tree of owner-signed blocks rooted at genesis. A block's slot is one greater than its parent's slot; slots are account positions, not times. Blocks of one account conflict when neither is an ancestor of the other. An owner equivocates by signing conflicting blocks; safety covers such owners, liveness does not promise a winner under persistent contention.

A closed checkpoint is  $S_e = (L_e, \Sigma_e)$ . The ledger  $L_e$  identifies finalized prefixes, retained unresolved branches, and lock records. A lock record names its block, its strength (notarization or recovery), and the epoch whose potential finality it protects; a certificate in an unrelated epoch cannot silently discharge an older lock. The application state  $\Sigma_e$  contains balances, received-send identifiers, and applied-operation identifiers. A block is final either from an eligible account FC/FF or because a decided checkpoint promoted it under Section 5.5; an undecided candidate has no finality effect.

Let  $F_e$  be the finalized block set of  $L_e$ . A validator also maintains a certified live overlay  $O_i$  of locally finalized blocks not yet in  $F_e$ . Its effective final set is  $F_e \cup O_i$ , and its live application view is the deterministic replay of that set. Section 5.6 defines installation without rolling back or reissuing effects. A deterministic function derives the committee  $C_e$  from  $\Sigma_e$ , never from a private overlay.

### 3.4 Latent finality and progress assumptions

**Definition 3.1** (Latent and late finality). For a fixed epoch and admission context, a block is latent-final at a cut if released signatures and valid dependency evidence suffice to verify an eligible FC or FF for it or for a descendant selecting it. Late finality is such a proof assembled or exposed after reports freeze or after a checkpoint is decided. A closure obligation also covers a certificate completed by subsequent old-epoch signatures from identities that have not stopped.

Preservation requires retaining the selected branch or a compatible finalized extension. Checkpoint promotion is permitted only for a uniquely retained prefix already protected by closing-epoch notarization.

Safety has no message-delay bound. Progress requires finite relevant admitted work, fair processing and retry, retained-data availability, and recurring sufficiently long bounded-delay windows in which  $\delta$  bounds delivery and processing and clock rates lie in  $[1 - \rho, 1 + \rho]$ . Table 2 maps premises to guarantees. An uncontested continuation has no conflicting owner-signed alternative at any new position, including signatures created earlier.

### 3.5 The two abstract services

RAI commits to sets by their roots and agrees on candidates by their digests. Two services turn these commitments into

shared state. RAI states their interfaces and the properties it uses; it does not fix their mechanics.

**Convergence.** A reporter calls  $\text{Freeze}_i(\text{ctx}_e, T_i, G_i)$ , which returns the signed header  $R_i$  of Section 5.2 and retains the frozen sets. Any validator calls  $\text{Reconstruct}_j(R_i)$ , which returns  $(T, G)$  or is still pending. The service must provide:

**C1 (Integrity).** If  $\text{Reconstruct}_j(R_i)$  returns  $(T, G)$  at a correct  $j$ , then  $\text{Root}(T)$  and  $\text{Root}(G)$  are the roots signed in  $R_i$ . With collision resistance,  $(T, G) = (T_i, G_i)$  when  $i$  is correct, and a Byzantine  $i$  is bound to one pair per header.

**C2 (Termination).** If a correct  $i$  froze  $R_i$  and  $R_i$  reaches a correct  $j$ , then under the data service and fair retry  $\text{Reconstruct}_j(R_i)$  eventually returns.

Full transfer of the committed sets, an authenticated difference against a shared projection, and set reconciliation all satisfy C1 and C2; they differ in cost. A Byzantine reporter can make only its own report unreconstructible.

**Consensus.** For handoff  $e$ , members of the closing committee  $K_e$  call  $\text{Propose}(\text{ctx}_e, v)$  with candidates of the form  $v = (Q, \mu, d)$  of Section 5.4, and  $\text{Decide}(\text{ctx}_e)$  returns a value  $v$  with a proof  $\pi_e$  that  $\text{Verify}(\text{ctx}_e, v, \pi_e)$  accepts at any party knowing  $K_e$ . RAI supplies the predicate  $\text{Eligible}_e(v)$  of Section 5.4, which a correct member can evaluate once it has reconstructed the reports  $v$  names. The service must provide:

**A1 (Agreement).** Two accepted proofs for  $\text{ctx}_e$  carry the same  $v$ .

**A2 (Eligibility).** An accepted proof for  $\text{ctx}_e$  carries a  $v$  with  $\text{Eligible}_e(v)$ . Equivalently, a correct member supports a value only after checking  $\text{Eligible}_e$  on reconstructed inputs, and no decision forms without correct support.

**A3 (Integrity).** A proof cannot be replayed for another context, and  $\text{Verify}$  needs only  $K_e$  and the retained inputs  $v$  names.

**A4 (Termination).** If at least  $N - f$  correct members of  $K_e$  eventually hold some eligible candidate for  $\text{ctx}_e$ , then every correct member eventually decides, given recurring prepared windows long enough for the service's own rounds.

A2 is external validity with a predicate the service treats as opaque. The members that evaluate it need reconstructed reports, which is where C2 feeds A4: convergence makes correct reports usable, usable reports make an eligible candidate constructible at every correct member, and the consensus service then decides it. Nothing in A1–A4 refers to accounts, blocks, or report contents; any Byzantine agreement protocol with external validity and verifiable decisions, leader-based or not, fits the interface.

## 4 Account-Local Processing

### 4.1 Blocks, operations, and validation

A block is  $B = (a, h_p, x, \sigma_a)$ : account, parent hash, ordered operation batch, and owner signature. Its identifier is

$$h(B) = H(\text{Block}, a, h_p, x).$$

The epoch is deliberately absent from this identifier, allowing a nonfinal omitted block to be retried with new-epoch votes. An empty batch is permitted for owner-authorized branch continuation. The two operations are  $\text{Send}(b, z)$ , for destination  $b$  and positive amount  $z$ , and  $\text{Receive}(s)$ , where  $s = (h(B), j)$  identifies a send at operation index  $j$ .

A send debits its owner. A receive credits only the named destination and requires an explicit proof that the send is already final. Transfer completion requires a final receive; the protocol does not create receives for passive recipients. These are separate owner-authorized operations, not an atomic update of two accounts.

Validation replays the entire unresolved selected path from its finalized anchor, then the proposed batch. It checks signatures, consecutive parent slots, destinations, positive amounts, balances, and duplicate receives using a temporary balance and received-send set. Checking each block only against the last finalized balance would miss a duplicate receive within an unresolved chain. Validation has no application effect. Every finalization repeats these checks and applies only previously unapplied operations.

A finality proof is either an account-certificate proof with the committee history, votes, attachment witnesses, and dependencies needed to verify eligibility, or a verified decided checkpoint that promoted the block. Proofs are finite, recursively verifiable evidence graphs; a circular reference is not a justification. A proof used for epoch- $e$  voting cannot depend on  $S_e$  or a later checkpoint. A receive's send proof must be available before a correct validator supports the receiving path. These conditions make replay dependencies well founded (Lemma 6.6).

### 4.2 Single-support voting and durable state

An account instance is  $(\text{session}, e, a, v)$ . A correct validator first-votes at most once in that instance, for the first admissible available block it accepts. Its first vote is also its notarization support. The only other notarization support is the late notarization below, cast in an epoch the validator has left; there is no second-look support for a rival and no account timeout vote. A correct validator supports at most one block per instance, by a first vote or a late notarization.

A block is *complete* in epoch  $e$  when its data and required ancestry are available and the validator has assembled its epoch- $e$  NC from  $q$  first votes. A complete block that is eligible is durably recorded as notarized before the validator final-votes, uses it as an eligible complete parent, or freezes its epoch report. The validator can final-vote only for its own first-voted block. It atomically records a terminal flag

**Table 2.** Assumption-to-guarantee map. None of the progress premises is a throughput or resource-isolation guarantee.

| Premise                         | Required condition                                                                                                                     | Guarantees using it                                                                     |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Safety model                    | Static Byzantine identities; at most $f$ per committee; authentication; persistent voting; valid ancestry and prefix-wide eligibility. | Account exclusion and cross-epoch safety.                                               |
| Convergence C1, consensus A1–A3 | Reconstructed sets match their signed roots; one eligible value decided per context, verifiably.                                       | Deterministic construction at every correct validator; identical installed checkpoints. |
| Data service                    | Correct holders retain and eventually serve blocks, votes, frozen reports, and committed candidate inputs.                             | Report usability and successor installation.                                            |
| Finite preparation              | Relevant admitted closing-epoch evidence is finite; reconstruction and validation are fairly scheduled.                                | Eventual reconstruction and finite BuildState.                                          |
| Convergence C2, consensus A4    | Correct reports reconstruct; a held eligible candidate is decided in a prepared window with $N - f$ responsive correct old members.    | Handoff termination.                                                                    |
| Account progress                | Available dependencies; common-open ready intervals; persistent uncontested submission with retry or fresh-child continuation.         | Account finality and eventual check-point inclusion.                                    |

with that signature; it does not wait to observe an FC before remembering that voting ended.

An eligible block finalizes on  $q$  final votes or  $N - p$  first votes. Its selected unresolved ancestors finalize with it, and incompatible provisional branches are removed from the effective live view. An NC alone has no balance effect. Learning eligible finality closes the resolved voting instances, but relaying blocks and votes continues. A failed early attempt never resets an instance’s first vote or terminal flag.

**Cross-epoch voting record.** A correct identity stops epoch- $(e - 1)$  first and final voting before opening epoch  $e$ . While  $S_{e-1}$  is unknown, it does not first-vote for a different block at a position where it supported a block in epoch  $e - 1$ . Re-voting the same block is allowed. Once the predecessor is installed, its finalized history, frontier, and carried locks govern the position. This guard prevents a common identity from changing sides during overlap; the core protocol does not assume enough common identities to make that guard alone a quorum argument.

**Late notarization.** A member of  $K_{e-1}$  keeps notarizing for its closing epoch until it installs  $S_{e-1}$ . When it first-votes  $B$  at  $(a, v)$  in epoch  $e$  while  $S_{e-1}$  is unknown, it also signs a notarization-only epoch- $(e - 1)$  vote for  $B$ , unless it supported a different block at  $(a, v)$  in epoch  $e - 1$ ; if it first-voted  $B$  itself there, that vote already notarizes. The late vote is durably recorded with its instance before release, like a first vote. It counts toward an epoch- $(e - 1)$  NC and toward nothing else: no FF or FC counts it, no final vote follows it, and no report contains it, since reports are frozen before it is cast. Late notarization gives a block published after the boundary the closing-epoch NC the overlap exception below requires; without it, such a block never has one.

### 4.3 Attachment and prefix-wide eligibility

Let  $d_a(S)$  be the maximum depth of any block for account  $a$  retained in checkpoint  $S$ , including unresolved branches. With  $S_{e-1}$  installed, a new epoch- $e$  root starts at depth  $d_a(S_{e-1}) + 1$  and extends a maximum-depth retained tip. Subsequent children require complete epoch- $e$  parents. An extension must respect finalized history and every carried lock on its selected ancestry. A shorter retained branch cannot reopen a position below the frontier.

Before  $S_{e-1}$  is known, first voting may prepare work from a maximum-depth tip of the latest closed ledger, from a closing-epoch finalized parent with its complete proof, or from a complete current-epoch parent. A closing-epoch notarized block may itself be re-voted when its parent is admissible. Such a block must also become complete in the current epoch before being used as a current-epoch complete parent. The signed admission-witness digest fixes the anchor and old evidence actually used; later rechecking cannot invent an earlier justification.

Ordinary early work is provisional. When the predecessor arrives, it is rechecked against the decided frontier, consecutive current-epoch parents, finalized history, and carried locks. A tally of fast-final size does not bypass this check. A notarization lock constrains settlement to its branch. A recovery lock can be discharged by a verified conflicting NC in the very epoch whose possible finality that lock protects: account exclusion then proves that its latent final certificate cannot exist. The discharge evidence is retained. A newer NC without this provenance is insufficient.

**The core overlap exception.** A block may finalize before  $S_{e-1}$  is known only if the verifier checks its entire

selected prefix against the latest closed state and proves cross-boundary exclusion at *every unresolved position being finalized*. Each such block, including the certificate target, must have a valid closing-epoch NC and a complete current-epoch NC for the same hash. The closing-epoch NC may combine first votes released before the boundary and late notarizations. A closing-epoch finalized prefix is already a safe anchor. Any inherited recovery lock bypassed by this path requires the matching-domain discharge proof above; inherited notarization locks cannot be bypassed.

Consequently, a descendant certificate can settle the selected prefix during overlap only when that descendant and all intervening unresolved blocks satisfy the same guard. A fresh descendant with no old-domain exclusion evidence, that is, without its own closing-epoch NC, remains provisional, even if its parent is notarized in both epochs. Otherwise disjoint committees could certify different children of that parent. This restriction sacrifices some overlap rather than deriving a cross-epoch exclusion result from a same-position parent certificate.

If the predecessor arrives first, nonfinal early work falls back to ordinary rechecking. Already finalized overlap work remains in the certified live overlay; installation cannot invalidate it. A decided checkpoint may promote only a uniquely retained closing-epoch notarized prefix; recovery-only uniqueness remains a lock. The supplement describes a possible alternative exclusion witness for fresh work, separately from this core rule.

#### 4.4 Fresh-child recovery

If a checkpoint retains unresolved maximum-depth tips after the promotion rule, the owner can extend a permissible tip with a fresh child. Finalizing that child selects its unresolved ancestors rather than reopening their slots. A notarization or recovery lock still constrains the branch choice; the owner cannot use a child to evade an undischarged obligation.

This continuation policy is also needed for liveness. A block retained at the new frontier may no longer be a legal new root at its old position. Persistent submission therefore means retrying an omitted block, or supplying a fresh, possibly empty, uncontested child of a retained admissible tip. Repeatedly resubmitting the same locked block is not by itself a progress guarantee. Existing owner-signed conflicting children can still prevent the continuation from being uncontested.

## 5 Pipelined Epoch Transitions

### 5.1 Lagged committees and local boundaries

The account committee for epoch  $e$  is  $K_e = C_{e-2}$ . That outgoing committee decides  $S_e$ ; the already known successor  $C_{e-1} = K_{e+1}$  verifies and installs it. The state  $S_e$  derives  $C_e$ ,

| Account epoch  | $e$       | $e + 1$            | $e + 2$           |
|----------------|-----------|--------------------|-------------------|
| Voters         | $C_{e-2}$ | $C_{e-1}$          | $C_e$             |
| Role for $S_e$ | Decide    | Verify and install | Selected by $S_e$ |

**Figure 2.** Committee roles are staggered. Closure of  $e$  overlaps account epoch  $e + 1$ ; the table imposes no wall-clock boundaries.

whose account service starts in epoch  $e + 2$ . Figure 2 separates selection, installation, and account voting. Genesis supplies  $S_{-1} = S_G$  and  $C_{-2} = C_{-1} = C_G$ .

After local duration  $D$ , a validator may end epoch  $e$  only when no older epoch remains closing. It stops old first and final voting, records all complete eligible old blocks as notarized, freezes its report against the closed  $S_{e-1}$ , and opens  $e + 1$ . Learning a decided checkpoint also stops any remaining signing for its closed epoch. Catch-up verifies predecessors in order and never reopens old vote records. A validator signs nothing new in an epoch it has left except late notarizations, which stop when it installs the checkpoint; it re-broadcasts what it signed before, so that certificates of the closing epoch can still complete from released votes.

There is at most one closing epoch per validator. If closure outlasts  $D$ , the next local boundary is deferred; successor voting continues, subject to eligibility. The two-epoch lag gives future members an opportunity to synchronize, not an upper bound on state transfer. Boundaries are local events: validators reach them at different times, and nothing in the protocol requires a synchronized stop.

### 5.2 Immutable signed reports

Every correct outgoing validator  $i$  signs one header

$$R_i = \text{Sign}_i(\text{ctx}_e, \text{Root}(T_i), \text{Root}(G_i)),$$

where the context binds the session, closing epoch, predecessor checkpoint, outgoing committee, and reporter identity. A Byzantine identity may equivocate, but a candidate selects at most one of its headers.

Let  $L_i$  be the reporter's ledger at freeze time. Its canonical tagged set is

$$T_i = \{(h(B), s_i(B)) : B \in L_i\}, \quad s_i(B) \in \{R, N, F\}.$$

An R entry is inherited recovery-protected state. An N entry has represented notarization but no locally observed finality. An F entry is final by inherited checkpoint state or by an explicit account certificate for the block or a descendant selecting it. The strongest status wins:  $F > N > R$ . Each entry also names the block's account position and parent, so that a selection places every block it names without needing the block body. Let  $V_i^{(e)}$  contain the hashes of all blocks for which  $i$  released an epoch- $e$  account vote before freezing.

The second committed set is

$$G_i = V_i^{(e)} \setminus \text{keys}(T_i).$$

The roots commit to ledger status and to voted-but-not-ledger membership, not to blocks or certificate objects. Correct reporters retain the frozen sets, vote records, and admission evidence. A later vote arrival, certificate assembly, or successor finalization can change their live view but cannot change the signed report. Late notarizations (Section 4.2) are released after freezing, so they are in no  $V_i^{(e)}$  and never construction input.

### 5.3 Usable reports

A root does not enumerate its members, so a validator that holds every block body still does not know which tagged entries a reporter froze. Reconstruction is a protocol precondition, and it is what the convergence service of Section 3.5 provides:  $\text{Reconstruct}_j(R_i)$  yields  $(T_i, G_i)$  with integrity C1. RAI adds the semantic check. A report is *usable* at  $j$  when both sets are reconstructed, every entry is well formed, and its claims are justified by evidence  $j$  holds: an R entry matches predecessor recovery state; an N entry has the relevant valid NC or an inherited notarization record; an F entry has valid inherited checkpoint finality or an explicit account certificate, possibly assembled in an earlier retained epoch; and a  $G_i$  membership is backed by  $i$ 's own signed first vote. Relaying another validator's vote never counts as reporter support. A report naming a block above the first position without its parent, unless the predecessor finalized that block there, is malformed; such a report is never selected, so one Byzantine report cannot make every candidate unconstructible. Missing evidence is fetched from retained data; a Byzantine report with unavailable or malformed material stays unusable. Closure needs only  $N - f$  usable reports, and at least that many reporters are correct.

### 5.4 Eligible candidates

A candidate selects exactly  $N - f$  usable report headers from distinct members of  $K_e$ , in canonical identity order, forming  $Q$ . It commits to an evidence manifest  $\mathcal{M}$  listing content-addressed blocks, report reconstructions, exact supporting vote records, and finite admission and dependency witnesses; the bytes are retained protocol data, and the manifest introduces no trusted object. The candidate value is

$$v_e = (Q, \mu_e, d_e), \quad \mu_e = H(\mathcal{M}), \quad d_e = H(S_e),$$

with  $S_e = \text{BuildState}(S_{e-1}, Q, \mathcal{M})$ . RAI's eligibility predicate, which the consensus service enforces through A2, is

$$\text{Eligible}_e(v) \iff \begin{cases} Q \text{ has } N - f \text{ usable reports of distinct members of } K_e, \\ H(\mathcal{M}) = \mu, \quad \text{BuildState}(S_{e-1}, Q, \mathcal{M}) = S \neq \perp, \end{cases}$$

All contexts, report identities, vote domains, and predecessor references must verify. A correct member evaluates

$\text{Eligible}_e$  only on reconstructed and validated inputs; receiving a digest establishes neither availability nor validity. The manifest removes an ambiguity between a committed report *claim* and the evidence later chosen to validate it: witness bytes are fixed before voting, so the state digest is a function of fixed semantic inputs rather than of certificate arrival order. A proposal that extends a certified candidate inside the consensus instance inherits  $Q$  and both digests; it cannot replace the report selection. The manifest is not a license to omit required report material, nor may it promote an N or  $G_i$  entry to final because an incidental certificate is present.

### 5.5 Deterministic construction

Let  $\text{Anc}(X)$  include  $X$  and all its account ancestors. A candidate block is an inherited block, a selected tagged block, or a block named by a selected  $G_i$ , with the ancestors and fixed dependencies needed to validate it. Eligibility is checked against the closed predecessor or the explicit overlap evidence retained with its admission; an unexplained speculative path is excluded from new recovery counts. Recovery admissibility checks anchor, ancestry, dependencies, and cross-boundary witnesses, but not the candidate's own certificate, which would make Rule 2 circular.

For a block  $B$ , let  $f_Q(B)$  mean that a selected F entry explicitly names  $B$  and its finality witness verifies, and  $n_Q(B)$  that selected N evidence represents an eligible notarization. The fresh reporter support set is

$$U_Q(B) = \{i : h(B) \in G_i \text{ and } \mathcal{M} \text{ verifies } i\text{'s first vote for } B\}.$$

Inherited R entries add no fresh support. Finality witnesses may be for descendants, but only the named block and its selected ancestors are final-output seeds.

**Final closure.** Start with inherited finality and all blocks satisfying  $f_Q$ . Take account-ancestor closure and recursively include the explicit finalized-send dependencies required by those final blocks; every added send has a verified finality proof fixed by the signed receive. Call the result  $F^*$ .

**Rule 1: represented notarization.** At each new account position, after excluding conflicts with  $F^*$ , retain the unique eligible represented NC target and its ancestry as a notarization lock. Same-domain account exclusion rules out two conflicting eligible NCs. The lock constrains continuation, not balances.

**Rule 2: reporter first votes.** If there is no represented eligible NC at that position and exactly one eligible block has  $|U_Q(B)| \geq r$ , retain that block and its ancestry as a recovery lock. Multiple threshold candidates cause neither an arbitrary winner nor certificate-free finalization; Theorem 6.4 shows that a formable fast-final block cannot coexist with a threshold rival.

**Algorithm 1** BuildState( $S_{e-1}, Q, M$ )

- 1: Verify the predecessor, selected identities, report roots, and the committed manifest.
- 2: Take every selected  $(T_i, G_i)$  from Reconstruct; verify the material for all required memberships and claims.
- 3: Form the parent-closed candidate set; classify admission and eligibility from fixed evidence.
- 4: Compute  $F^*$  by explicit-finality, ancestor, and finalized-send dependency closure.
- 5: Compute represented NC targets and reporter support sets for every position before pruning.
- 6: Check inherited locks and matching-origin discharge witnesses; reject incompatible claims.
- 7: Apply Rules 1–2; retain inherited survivors and the ancestor closure of required lock targets, forming  $R_Q$ .
- 8: Apply Rule 3 to obtain  $P_Q$ ; compute  $F^+$  from  $F^* \cup P_Q$ .
- 9: Retain  $R_Q \setminus F^+$  and replay  $F^+ \setminus F_{e-1}$  once in canonical dependency order.
- 10: Return  $(L_e, \Sigma_e)$ ; Rule 3 has effect only if this candidate is decided.

**Inherited state and discharge.** Retain predecessor unresolved survivors unless explicit compatible finality excludes them or a matching-origin NC proves a recovery obligation impossible. Check lock discharge before permitting a conflicting new branch; never infer it from a certificate’s newer epoch number. All evidence summaries are computed before any omission. Let  $R_Q$  be the surviving inherited unresolved state, selected lock targets, and their ancestor closure after finality and valid discharge.

**Rule 3: checkpoint promotion.** For each account, starting at the tip of  $F^*$ , promote the next block only while it is the unique child retained in  $R_Q$  and the manifest verifies a closing-epoch NC for it; stop at the first fork or block without such an NC. The resulting prefix  $P_Q$  is candidate-final until the consensus service decides the candidate. Recovery-only blocks are never promoted. Let  $F^+$  be the ancestor- and finalized-send-dependency closure of  $F^* \cup P_Q$ ; the output ledger marks  $F^+$  final and retains  $R_Q \setminus F^+$ .

Replay orders parents before children and finalized sends before dependent receives; among independent ready blocks it chooses the least  $(a, v, h(B))$ . A promoted receive uses the send-finality witness fixed at admission and cannot depend on the checkpoint under construction. A cyclic or semantically invalid graph is rejected; Lemma 6.7 shows that an all-correct usable selection is never rejected. Algorithm 1 fixes the order of validation, summarization, lock selection, and replay.

### 5.6 Installing a decided checkpoint

Let a validator’s current closed base be  $S_b$  with finalized set  $F_b$ , and let  $O_i$  be its certified overlay, so that its effective final set is  $J_i = F_b \cup O_i$ . On an accepted proof  $\pi_e$  for  $v_e$ , the

validator recomputes  $S_e$  from the inputs  $v_e$  names, checks  $H(S_e) = d_e$ , and installs it with final set  $F_e$ :

$$J'_i = J_i \cup F_e, \quad O'_i = J'_i \setminus F_e.$$

The union is checked for account compatibility, final dependencies, and valid lock discharge. The validator adopts  $S_e$  as its base and materializes the live view as  $\text{Replay}(\Sigma_e, O'_i)$ . This rebases a certified set of effects; it does not authorize old sends again, and persistent operation identifiers suppress duplicate application and duplicate external notifications.

The installation invariant has three parts. The base’s final set grows monotonically and may include checkpoint-promoted blocks. Every overlay block has an explicit account-finality proof for itself or a selecting descendant, and the union with the base is dependency-closed. The union extends each base-final account prefix and respects every undischarged notarization or recovery obligation; a recovery lock superseded by a matching-domain exclusion witness leaves the effective view without editing the signed checkpoint.

Committee  $C_{e-1}$  holds no second election. It verifies the outgoing committee’s proof, reconstructs the state, derives  $C_e$  from the canonical  $\Sigma_e$ , and rechecks nonfinal early work. Additional live finality never changes that committee selection. Invalidated provisional work is discarded without clearing its persisted first vote or terminal state; already finalized overlap work remains certified and applied. Locks the checkpoint retains are continued, not reopened: a notarization lock gets an instance in the open epoch, a recovery-only lock once its block holds a valid closing-epoch NC, and a rival at a retained position is refused.

## 6 Correctness

The arguments concern the core protocol, including the prefix-wide overlap guard, late notarization, and Rule 3 promotion, and they are relative to the services of Section 3.5: safety uses C1 and A1–A3, termination additionally uses C2 and A4. They do not include the optional fresh-work extension of the supplement. Cryptographic guarantees hold except with negligible probability.

### 6.1 Safety

**Lemma 6.1** (Account exclusion). *In one account instance, at most one block has an NC, whether it counts first votes, late notarizations, or both. An FC or FF excludes a conflicting NC. If an FF is formable, every rival has at most  $f+p < r$  first-vote supporters.*

*Proof.* Two  $q$ -quorums intersect in at least  $2q - N = f + 1$  identities, one of them correct, which supports one block only in the instance, by a first vote or a late notarization (Section 4.2). An FC has a correct final voter who first obtained an NC; an FF contains  $N - p \geq q$  first votes. For the last claim, with actual Byzantine count  $f' \leq f$ , an FF has

at least  $N - p - f'$  correct first voters, so at most  $p$  correct identities remain for rivals; adding every Byzantine identity gives at most  $p + f' \leq p + f$ .  $\square$

**Lemma 6.2** (Late notarization adds no old finality). *Every epoch- $e$  FC or FF that can be formed consists of correct votes released before their signers froze, together with Byzantine votes.*

*Proof.* A correct validator releases no epoch- $e$  first or final vote after leaving epoch  $e$  (Section 5), and neither certificate counts a late notarization: an FF counts first votes, an FC final votes.  $\square$

**Lemma 6.3** (Frozen-report exposure). *A correct reporter's released final vote implies a frozen N or F entry for its selected block. Its first vote implies a frozen ledger entry for that block or a membership in  $G_i$ . These statements persist after report creation.*

*Proof.* Final voting follows durable eligible notarization, and account exclusion prevents incompatible same-domain finality from removing the selected path.  $G_i = V_i^{(e)} \setminus \text{keys}(T_i)$  partitions the reporter's old-domain voted hashes. Stopping precedes freezing; roots and tags are then immutable, and C1 ensures that what a correct validator reconstructs is exactly what was frozen.  $\square$

**Theorem 6.4** (Closure preserves latent and late finality). *Fix a valid closed predecessor. If an eligible old-epoch account final certificate can be formed, every eligible candidate (every  $v$  with  $\text{Eligible}_e(v)$ ) preserves its selected branch and ancestors, as compatible finality or as the locks that certificate requires. The certificate may be assembled after report freeze or after the checkpoint is decided.*

*Proof.* Let  $R$  be the selected reporters. For a possible normal certificate with final-voter set  $A$ ,  $|(R \cap A) \setminus \text{Byz}| \geq q - 2f = p + 1$ ; each such correct reporter froze an N or F entry, where F exposes finality and N triggers Rule 1. For a possible fast certificate,  $|(R \cap A) \setminus \text{Byz}| \geq (N - p) - 2f = r$ ; each of these reporters exposes the block through  $T_i$  or through  $G_i$  with its verified first vote, and in the latter case Rule 2 retains it. Lemma 6.1 prevents valid discharge or a threshold rival from eliminating genuine fast finality. Usability and the committed manifest make these memberships available to BuildState. Later unselected or Byzantine signatures do not remove the required correct intersections.  $\square$

**Lemma 6.5** (Checkpoint-promotion safety). *A Rule 3 promoted block cannot conflict with an eligible closing-epoch NC, FC, or FF, so a decision that finalizes the promoted prefix is compatible with old-domain finality and with core overlap finality.*

*Proof.* Every promoted block has a verified closing-epoch NC; Lemma 6.1 excludes a conflicting old NC and hence conflicting old FC/FF. Core overlap finality for a conflicting

successor block would require a conflicting closing-epoch NC at the first divergent position, also impossible. Promotion stops at retained forks and never promotes recovery-only evidence.  $\square$

**Lemma 6.6** (Acyclic final dependencies). *For any finite set of valid final blocks closed under account ancestors and required finalized sends, parent and send-to-receive edges form a directed acyclic graph.*

*Proof.* Order blocks by the earliest event at which valid final evidence justifies them. Along a parent edge this event cannot become earlier at the child; along a send-to-receive edge it is strictly later at the receive, because admission required the send's finality proof and Rule 3 cannot supply it for the checkpoint under construction. A cycle cannot consist only of parent edges, since slots increase strictly, so it contains a send-to-receive edge and a strict increase around a cycle, a contradiction.  $\square$

**Lemma 6.7** (Deterministic construction). *Assume the predecessor satisfies the closure invariant. For fixed usable selected reports and a valid immutable manifest, BuildState returns one canonical candidate state at every correct validator. It preserves inherited finality, carries all mandatory surviving locks, and applies only explicit account finality plus Rule 3 promotion. All-correct usable inputs yield a finite valid construction.*

*Proof.* By C1 every correct validator reconstructs the same  $(T_i, G_i)$  for each selected header, and the manifest fixes witnesses and proof choices; hence explicit finality and  $R_Q$  are the same everywhere. Lemma 6.1, predecessor validity, attachment, and Theorem 6.4 make the retained finality and locks compatible. Rule 3's tests fix  $P_Q$ , safe by Lemma 6.5; Lemma 6.6 and the canonical tie-break fix replay. Malformed Byzantine inputs are excluded by usability; all-correct usable inputs yield a finite state.  $\square$

**Theorem 6.8** (Transfer safety across epochs). *Assuming C1 and A1–A3, finalized histories at correct validators are compatible and irreversible across all core-protocol epochs; finalized balances are nonnegative, and each send is received at most once.*

*Proof.* Induct from genesis. Within an epoch, Lemma 6.1 excludes incompatible account finality. At a boundary, A2 ensures that the decided  $v_e$  is eligible, so by Theorem 6.4 it preserves every formable old final branch and by Lemma 6.5 its promotion is compatible; A1 and A3 ensure that every correct validator installs the state of the same  $v_e$ , which by Lemma 6.7 is the same state. For overlap, the first divergent unresolved position on an early successor-final prefix has a closing-epoch NC for the successor block by the prefix-wide guard, possibly assembled from late notarizations, which by Lemma 6.2 add no old finality that Theorem 6.4 would have to preserve; a conflicting closing-epoch

certificate or a conflicting Rule 3 promotion would require a conflicting NC, contradicting Lemma 6.1. Matching-origin discharge and inherited locks remain enforced after installation, and the base/overlay union with applied-operation identifiers prevents rollback and duplicate application. Replay checks balances and finalized-send dependencies.  $\square$

The proof uses nothing about how the consensus service reaches its decision: an eligible value is safe whichever one is chosen, because Theorem 6.4 is universal over eligible candidates. This is what lets RAI leave the service abstract.

## 6.2 Conditional termination

**Lemma 6.9** (Correct reports become usable). *Under the data service, finite relevant admitted work, fair processing, and C2, every correct report eventually becomes usable at every correct validator.*

*Proof.* A correct reporter stops old first and final voting, so its old vote set and roots are finite, and its report is well formed. By C2 every correct validator eventually reconstructs  $(T_i, G_i)$ . The data service delivers the blocks, ancestry, signed votes, and witnesses behind its claims, and fair processing completes the finite validation. A withholding Byzantine reporter can make only its own report unusable;  $N - f$  correct reports remain.  $\square$

**Theorem 6.10** (Handoff termination). *Assume the data service, fair processing, C2, A4, and recurring prepared windows with  $N - f$  responsive correct members of  $K_e$  long enough for reconstruction, candidate preparation, and the consensus service’s rounds. Every enabled handoff is eventually decided and, with eventual retained-data delivery, installed by correct successors.*

*Proof.* By Lemma 6.9,  $N - f$  correct reports become usable at every correct member, which by Lemma 6.7 can then construct an eligible candidate; the canonical selection order makes correct members that hold the same usable reports construct the same one. A4 then decides some eligible value in a sufficient window; A3 lets successors verify it, and retained authorizing data let them recompute and install the state, which enables the next deferred boundary.  $\square$

**Theorem 6.11** (Uncontested account continuation). *Assume handoff termination, available final-send dependencies, and recurring common-open ready intervals. An owner following the retry-or-fresh-child policy of Section 4 eventually finalizes each target of a finite valid uncontested continuation, provided each required new position has such an interval, and the finalized targets eventually enter a checkpoint.*

*Proof.* In a ready interval,  $N - f \geq q$  correct first votes create an NC and the same validators can final-vote. An attempt interrupted by a boundary is preserved by Theorem 6.4; an omitted block can be retried, a uniquely retained notarized target may finalize at the handoff, and a remaining

retained fork can be selected by a fresh uncontested child. Repeating this over the finite continuation and using eventual evidence delivery gives finalization and checkpoint inclusion.  $\square$

With ancestry prepared, normal finality takes three delivery stages after broadcast: block, first votes, final votes; an available  $N - p$  first-vote fast quorum takes two. A sufficient account interval budget is

$$\frac{D}{1 + \rho} > \kappa_A + R_A + 3\delta, \quad (5)$$

where  $\kappa_A$  bounds opening skew and  $R_A$  bounds readiness after the last opening, including predecessor closure. Deferred boundaries only lengthen an open interval. The corresponding window for the consensus service depends on the instantiation; Section 9 gives the condition for ours. These are conditional communication bounds, not measured latency.

## 7 Costs, Scope, and Limitations

**Agreement is not total system work.** The agreed value contains  $N - f$  report headers and two digests, so its reference portion is  $O(N)$ , independent of the number of account blocks. The evidence manifest, reconstruction data, validation, and retained histories scale with old state, new votes, and unresolved branches, and the consensus service may need several rounds. One checkpoint per epoch means neither one agreement round nor constant reconfiguration cost.

**Account locality is not resource isolation.** A correct validator releases at most one first vote and one final vote per account instance. This bounds neither an adversary’s owner-signed alternatives nor network traffic, retained evidence, or validation work. All-to-all vote dissemination has quadratic recipient deliveries in committee size per vote stage.

**The pipeline has an explicit frontier.** Lagged committees permit preparation before closure, and the core early-settlement path requires old-domain exclusion throughout the newly finalized prefix. Late notarization supplies that evidence for fresh work: the outgoing committee keeps notarizing until it installs the checkpoint, at the cost of one more signature per block and member during each close. A block whose late notarizations split, or that lacks one of the  $q$  supporters, still waits for the predecessor checkpoint, so a long handoff delays it by the handoff’s duration; Section 9 shows this residue as a latency tail under forks. A current NC for a fresh child does not protect it against an old committee’s conflicting child; its own late closing-epoch NC does.

**Checkpoint promotion is narrow.** A boundary decision may finalize a uniquely retained closing-epoch notarized prefix without an account FC/FF. It never promotes recovery-only state and provides no general fork-resolution

service: a position whose votes split evenly stays retained, identically everywhere, until the owner extends it.

**Retention and progress are conditional.** Reports are compact commitments to potentially large data; reconstructing one can cost nearly a full ledger-hash set in the worst case, whatever the convergence mechanism. Forks and retained dependencies can accumulate without bound; the model supplies no admission-control or garbage-collection policy. Departing validators cannot erase handoff evidence until successors durably hold it.

**Membership and finality have different state scopes.** Committee derivation uses the checkpoint base, while an account’s live view can include additional certified finality. This prevents later evidence from changing a selected committee, but checkpoint-based membership does not instantly reflect all locally known finalized balances. RAI is for authenticated committees with an externally maintained fault bound, not a permissionless staking system.

**Security and expressiveness boundaries.** The static adversary excludes later compromise of departed correct validators; long-range security, mobile corruption, and forward-secure keys need separate designs. Account operations exclude shared objects, atomic multiwriter transactions, passive-recipient credit, and a guaranteed winner under arbitrary equivocation. The proofs are written arguments, not machine-checked verification.

## 8 Related Work

### 8.1 Payments without a global execution order

Guerraoui et al. characterize asset transfer’s consensus number, including the single-owner case [10]. FastPay provides prefunded Byzantine settlement without a global transaction order [1]. Astro builds asynchronous payments on broadcast [5]; Zef adds private coin payments and storage-conscious lifecycle mechanisms [2]. Nano provides account-local chains and distinct send and receive operations [14]. These precedents establish the application-level opportunity, not RAI’s transition rules; RAI does not claim that single-writer consensus avoidance or account-local ordering is new.

Astro’s extended version discusses asynchronous replica reconfiguration with view-tagged messages and account-log transfer [5, 9]. RAI’s distinction is the frozen-report evidence that preserves first- and final-vote obligations across a boundary while the next epoch is already voting.

### 8.2 Hybrid execution and contention recovery

Sui Lutris uses consensus for shared objects and checkpoint sequencing, including end-of-epoch handling of processed certificates [3]. RAI reconstructs account obligations from reports and does not continuously sequence ordinary certificates; there is no certificate sequence to drain before the

epoch ends, which is what makes the pipelined transition possible. Cuttlefish broadens the fast path with collective objects and uses FastUnlock to address contention [11]. RAI’s boundary promotion is narrower: it finalizes only a uniquely retained notarized prefix at reconfiguration.

### 8.3 Dynamic membership and certificate boundaries

Pastro studies permissionless asynchronous asset transfer with weighted stake and a dynamic adversary [12]; RAI assumes equal-weight authenticated committees and static corruption. Dynamic Byzantine Reliable Broadcast and asynchronous Byzantine reconfiguration address broader changing-participation settings [9, 13]. PDCC combines certificate-based payments with membership change and periodic settlement; its installation rule accepts certificates for the installed configuration and states a correct-member overlap assumption [16]. RAI instead preserves eligible old-domain certificates that may be assembled later. Table 3 summarizes the boundary mechanisms.

### 8.4 Agreement with external validity and set reconciliation

Consensus with external validity, where a predicate restricts decidable values, is a standard interface [4]; RAI uses it with the eligibility predicate of Section 5.4 and requires verifiable decisions so that the successor can install without voting. Any protocol meeting A1–A4 fits; Section 9 instantiates it with a leader-based voting construction [17, 18]. Likewise, authenticated set reconciliation is a mature building block [7, 8, 19]; RAI’s requirement is only that each frozen tagged ledger and voted-but-not-ledger set be reconstructible and checkable against its signed root. The preservation proof depends on those memberships and quorum intersections, not on a particular reconciliation or agreement mechanism.

## 9 Implementation and Evaluation

**Prototype.** We implemented RAI in RsNano [15], a Rust implementation of the Nano node [14]. Account elections follow Section 4 with the equal-weight thresholds of Table 1 ( $f = p = 1$ ,  $N = 6$ ) and the late notarization and prefix-wide overlap rule of Sections 4.2 and 4.3. Epochs end at fixed times  $T_0 + kD$  on every validator’s local clock. At its boundary a validator freezes and signs its report; the construction of Section 5.5 includes Rule 3 promotion. Every first vote, final vote, late notarization, left epoch, and frozen report is written to an LMDB environment of its own before its signature leaves the process, one write per batch of votes the voter releases, and a restarted node reloads these records and refuses whatever they forbid; the environment is separate from the ledger’s so that the voter never waits for the ledger’s single writer. Late notarizations are generated apart from the consensus instantiation’s votes, open no

**Table 3.** Selected boundary mechanisms. The comparison concerns the cited designs, not measured performance or a completeness ranking.

| Design         | Relevant mechanism                                                                                                                            | Distinction from RAI                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Astro [5]      | Asynchronous view changes and account-log transfer in the extended-version appendix.                                                          | RAI studies frozen-report quorum obligations and overlaps the next epoch’s voting with the close.                  |
| Sui Lutris [3] | Processed certificates enter consensus before end-of-epoch handover.                                                                          | RAI has no continuous account-certificate sequence to drain.                                                       |
| Pastro [12]    | Permissionless asynchronous asset transfer with dynamic participation.                                                                        | RAI’s membership and corruption assumptions are narrower.                                                          |
| PDCC [16]      | Checkpoint-installed configurations and current-configuration certificate acceptance.                                                         | RAI’s eligibility predicate preserves eligible late old-domain finality.                                           |
| RAI            | Immutable reports, eligible candidates over abstract consensus and convergence services, obligation locks, narrow notarized-prefix promotion. | Account certificates finalize online; a decided checkpoint can also finalize a uniquely retained notarized prefix. |

account instance at their receivers, and are kept apart from the vote records reports and manifests are built from. A position whose seen first votes are already split so that no block can reach  $q$  is not re-voted at the boundary, since re-voting the same blocks would repeat the split. The two services of Section 3.5 are instantiated as follows.

**Convergence by rateless set reconciliation.** A report’s two committed sets are reconstructed with rateless invertible Bloom lookup tables [7, 8, 19]. Each set element is a fixed 105-byte item naming account, height, hash, parent, and tag. A validator that needs a report starts from its own live projection of the closing epoch, which usually differs from the reporter’s frozen set in a few entries, and asks for coded symbols of the frozen set; any validator that holds the reconstructed set answers, so the reporter is not a single point of service. Batches start at 16 symbols and grow to half of what has been received, up to 500. Peeling the symbols against the local projection yields the symmetric difference; the requester applies it and checks the signed root, which gives C1. In the runs below, reconstruction needed 1.5 to 2.9 coded symbols per differing item and no stream was abandoned. Before a report is usable, the validator fetches, by hash, the signed votes behind every N and F entry and every residual record it does not already hold, and rechecks only the entries that were missing. A report naming a block above the first position without its parent is malformed and never selected.

**Consensus by a Kudzu-style election.** The closing committee decides the checkpoint with one leader-based voting instance in the style of Kudzu [18], whose rounds, vote kinds, thresholds, and proof arguments are given in the supplement. Leaders rotate through the committee in canonical key order; a leader proposes the candidate built from the canonical prefix of usable reports, so two correct leaders holding the same reports propose the same value and

the first round normally decides. A candidate carries the report selection, the state digest, and the manifest digest  $\mu_e$  over the evidence: for every block a selected claim rests on, the committee members whose signed first and final votes the leader used, per epoch, as bit sets over the committee in key order. A validator first-votes a proposal only after reconstructing its reports, recomputing the state hash, and committing to the same evidence, which is A2 on the eligibility predicate. A validator whose own votes yield the same digest has checked that evidence already; otherwise it fetches the manifest by digest in chunks, fetches the votes it lacks, and checks every claim against the manifest’s votes alone before voting. final votes with threshold  $q$  form the decision proof, which successors verify against the committee (A1, A3). A round whose leader is offline or Byzantine times out and the next leader proposes (A4). A sufficient local timeout condition for this instantiation is  $\Delta_E/(1 + \rho) > \kappa_E + \tau_E + \delta$ , where  $\kappa_E$  bounds round-entry skew and  $\tau_E$  bounds proposal preparation and validation [6]. Successors install a decided checkpoint off the vote-processing path and fetch, by hash, any block of a retained or finalized branch they do not hold.

**Departures from the protocol.** First, the manifest commits to signed votes; blocks and ancestry are placed by the report items themselves, and admission witnesses are not committed separately. Second, the consensus instantiation’s own votes, the close election’s first, final and timeout votes, are not persisted; durability of the account path and of reports is RAI’s, durability of the agreement service is the instantiation’s. Third, retained vote records keep a fixed cap of four epochs, since the reports of the next three epochs may still cite an epoch’s late finality; only the handoff evidence, an epoch’s frozen reports and their reconstructions, is released by acknowledgement, once members of the successor committee carrying  $N - f$  of its weight have installed the checkpoint. Fourth, an inherited recovery lock is discharged by explicit compatible finality only;

the matching-origin discharge witness of Section 5.5 is not implemented, so a position whose votes split stays retained until its owner extends it. Fifth, the epoch origin  $T_0$  is not persisted, so the harness cannot restart a validator within a run; the restart path is exercised by unit tests only.

**Baseline.** The baseline is unmodified RsNano (develop), which runs Nano’s open representative voting: a block is confirmed once final votes reach 67% of the online weight, after a first round of non-final votes. It has no epochs and no checkpoints. Final votes cannot be withdrawn, so a fork whose votes split can leave no side able to reach the quorum; the baseline gives no termination guarantee in that case, and its unresolved elections keep their slots in a bounded election container.

**Setup.** All validators run as separate processes on one MacBook Air (Apple M1, eight cores, 16 GiB, macOS 14.6) and communicate over loopback; LMDB runs without synchronous writes, so the durable records of Section 9 survive a process crash but not a power loss. The election container holds up to 20,000 instances; RAI keeps every unresolved fork instance as evidence until its owner extends the position, and a 10% fork workload leaves about 4,500 of them. Six equal-weight representatives hold the voting weight. A load generator funds 45,000 accounts and then publishes 45,000 blocks at an offered 2,000 blocks/s, each on the frontier of a randomly chosen account, so account chains grow during the run. The generator follows the owner policy of Section 4: it builds on a confirmed frontier, or, once an unconfirmed non-fork frontier has waited 5 s, extends it with the account’s next block as a fresh child; a position where it published a fork is never extended. With forks enabled, it publishes a conflicting sibling for 5% or 10% of the blocks, each sibling to half of the validators. An *offline* validator runs no node and never votes but keeps its weight. A *Byzantine* validator also runs no node; the generator votes with its key at random, choosing any vote kind for recently published blocks and contradicting itself. Both arms use the same load-generator source; the binaries differ only in the vote encoding, since RAI votes carry their epoch. RAI uses 8 s epochs, so a run spans three checkpoints. A run *settles* when every running validator holds the same cemented state and, for RAI, has decided the same checkpoints; a position left retained by a split vote counts as settled when it is uncemented on every validator alike. We report non-fork blocks only: goodput is the number of non-fork blocks confirmed divided by the time from the start of publishing to the last such confirmation, and latency runs from a block’s publication to its confirmation at one validator. The baseline’s publishing phase may last 1.5 times RAI’s longest publishing phase for the same variant; setup time is not counted. Each variant ran once per system on an otherwise idle machine; the harness refuses to start while any other process uses more than a quarter of a core. The baseline rows were

taken in an earlier pass on the same host with the same generator; the baseline code did not change. The RAI rows of Table 4 were taken in one pass on the build with late notarization; Table 5 gives two earlier passes of the same day.

**Without forks.** Table 4 shows both systems keeping up with the offered load; RAI’s goodput is 2 to 3% higher. RAI’s median latency is about half the baseline’s, 104 to 110 ms against 189 to 207 ms: an uncontested block finalizes on one round of  $N - p$  first votes, where the baseline needs a round of final votes after a quorum of non-final ones. RAI’s tails are also lower, p95 176 to 305 ms against 234 to 410 ms. Closes took 1.2 to 2.7 s when the first election round decided and up to 3.9 s when a round timed out, but blocks published meanwhile did not wait for them: 91 to 96% of the blocks late-notarized during a close reached a closing-epoch NC and finalized before the checkpoint, and 40 to 551 instances per boundary waited for the predecessor gate. Three checkpoints were decided on every running validator in all variants. The first two closes of every run decided in the first round; the third decided in the second round with one validator offline or Byzantine, when leader rotation reached the missing validator.

**With forks.** The baseline finished none of the six fork variants: forks whose votes split stay unresolved and occupy election slots, which delays the non-fork blocks queued behind them, and the confirmed share at the deadline ranged from 69% to 95%. RAI settled all six with three checkpoints decided identically on every running validator. At 5% forks its median stayed within 25 ms of the no-fork median and p95 was 0.35 to 1.3 s; at 10% forks the median rose to 141 to 187 ms, p95 to 0.8 to 2.1 s, and goodput fell to 1,546 to 1,627 blocks/s. Forks lengthen the closes: more positions are retained at each freeze, reports and manifests grow with them, and a close took 2.2 to 8.6 s at 10% forks. They also leave more fresh blocks without a late NC: 85 to 88% of late-notarized blocks reached one at 5% forks and 80 to 81% at 10%, the rest being split siblings and blocks one supporter short, which wait for the checkpoint; with one validator offline, the third close’s second round adds the leader timeout to that wait. Most fork positions stayed retained: the generator sends each sibling to half of the validators, so a fork’s first votes split three against three and neither side reaches  $q = 4$ ; without a notarization Rule 3 cannot promote either side, and the generator never extends a position where it equivocated. About 2,100 to 2,250 positions at 5% forks and 4,360 to 4,480 at 10% were uncemented when the runs settled, identically on every validator; they did not prevent settlement, and every non-fork block confirmed.

**Late notarization and the cost of the core rule.** Table 5 gives the same workload on the build of the same day without late notarization, and on an earlier build whose overlap rule included the predecessor-backed route of the

**Table 4.** Non-fork goodput (blocks/s) and non-fork confirmation latency (ms) at an offered 2,000 blocks/s, one run per variant. With forks the non-fork share of the offered load is 95% or 90%. CP is the number of checkpoints every running validator decided identically. A baseline run that did not finish is given by the share of the 45,000 blocks confirmed when its publishing deadline expired.

| Variant                | RsNano develop (no checkpoints) |     |     |     | RAI prototype (8 s epochs) |     |      |      |    |
|------------------------|---------------------------------|-----|-----|-----|----------------------------|-----|------|------|----|
|                        | Blocks/s                        | p50 | p95 | p99 | Blocks/s                   | p50 | p95  | p99  | CP |
| No forks               | 1901                            | 189 | 410 | 622 | 1,946                      | 104 | 305  | 519  | 3  |
| No forks, 1 offline    | 1907                            | 207 | 238 | 285 | 1,970                      | 110 | 176  | 270  | 3  |
| No forks, 1 Byzantine  | 1921                            | 205 | 234 | 289 | 1,980                      | 109 | 186  | 291  | 3  |
| 5% forks               | did not finish: 95.4% confirmed |     |     |     | 1,809                      | 117 | 1279 | 2274 | 3  |
| 5% forks, 1 offline    | did not finish: 90.4% confirmed |     |     |     | 1,817                      | 127 | 351  | 546  | 3  |
| 5% forks, 1 Byzantine  | did not finish: 93.0% confirmed |     |     |     | 1,827                      | 124 | 374  | 807  | 3  |
| 10% forks              | did not finish: 87.3% confirmed |     |     |     | 1,627                      | 141 | 813  | 1653 | 3  |
| 10% forks, 1 offline   | did not finish: 69.2% confirmed |     |     |     | 1,546                      | 187 | 2111 | 2857 | 3  |
| 10% forks, 1 Byzantine | did not finish: 86.4% confirmed |     |     |     | 1,550                      | 181 | 956  | 2963 | 3  |

**Table 5.** The RAI prototype on the same workload without late notarization (the core rule alone: every block published during a close waits for the checkpoint) and on an earlier build whose overlap rule included the predecessor-backed route of the supplement, one run per variant: non-fork goodput (blocks/s) and latency (ms).

| Variant                | Without late notarization |     |      |      | Predecessor-backed route (earlier build) |     |     |     |
|------------------------|---------------------------|-----|------|------|------------------------------------------|-----|-----|-----|
|                        | Blocks/s                  | p50 | p95  | p99  | Blocks/s                                 | p50 | p95 | p99 |
| No forks               | 1,942                     | 108 | 2403 | 3399 | 1,968                                    | 98  | 280 | 880 |
| No forks, 1 offline    | 1,978                     | 117 | 1432 | 1962 | 1,978                                    | 110 | 205 | 441 |
| No forks, 1 Byzantine  | 1,969                     | 117 | 1280 | 1897 | 1,978                                    | 112 | 189 | 433 |
| 5% forks               | 1,807                     | 133 | 4169 | 4940 | 1,824                                    | 108 | 611 | 859 |
| 5% forks, 1 offline    | 1,815                     | 126 | 2070 | 3013 | 1,836                                    | 117 | 245 | 552 |
| 5% forks, 1 Byzantine  | 1,750                     | 125 | 2191 | 2802 | 1,841                                    | 118 | 298 | 513 |
| 10% forks              | 1,440                     | 401 | 3915 | 4797 | 1,679                                    | 128 | 660 | 960 |
| 10% forks, 1 offline   | 1,267                     | 203 | 8188 | 8820 | 1,677                                    | 131 | 411 | 492 |
| 10% forks, 1 Byzantine | 1,253                     | 281 | 5287 | 8469 | 1,681                                    | 131 | 448 | 717 |

supplement. Without late notarization a block published during a close has no closing-epoch NC, so under the core rule every such block waited for the checkpoint: 1,900 to 7,100 instances per boundary, client confirmations stopped for one to three seconds at each close, and p95 was 1.3 to 8.2 s. Late notarization keeps the core rule and its proof and brings the no-fork tails to those of the predecessor-backed route, whose exclusion argument the main theorem does not cover: p95 176 to 305 ms against 189 to 280 ms. With forks the residue described above remains, p95 0.35 to 2.1 s against 0.25 to 0.66 s, and at 10% forks goodput is 1,546 to 1,627 against 1,677 to 1,681 blocks/s. Unlike the predecessor-backed route, late notarization needs no members common to the two committees: its exclusion is within the closing epoch’s own domain.

**Defects found by the matrix.** Several defects were found and repaired while measuring; none is a failure of the two services. First, a bookkeeping slip in the report: a validator records, for every block it votes for, the parent that block names, so that its residual records place the block; the record was dropped when the slot state of the older of two consecutive epochs that both voted the block was discarded. The newer report then committed a residual record with a zero parent, BuildState failed at every leader that selected it, and two variants never closed an epoch. The record now outlives every slot state that votes the block, and a report naming a block without its parent is malformed and never selected, so one Byzantine report cannot stall the close the same way. Second, a block whose first votes split across an epoch boundary, three cast in the epoch left and two in the next, was retained as a recovery lock and never finalized:

the validators that had voted it in the old epoch opened no instance in the new one, and nothing but an owner continuation resolves a recovery lock. Every unfinalized instance of the epoch left is now re-voted in the new epoch, which Section 4 permits, and the generator implements the owner policy for the positions that remain. Third, manifest chunk requests were at first flooded to every representative; each returned a reply per chunk and the inbound queues overflowed, so two validators took four seconds to fetch an epoch’s manifest and the round timed out. Requests now go to the proposal’s sender. Fourth, re-voting every unfinalized instance doubled the permanently open fork instances and filled the election container to its former cap of 5,000, after which no new block was activated on any validator; a carried instance at a retained position whose lock cannot continue is now discarded when the checkpoint installs, a re-voted recovery lock continues once it holds an open-epoch NC, and the cap was raised. Fifth, three costs of the handoff itself lengthened the wait at the gate: committee derivation deduplicated the epoch’s finalized blocks quadratically, 0.2 to 0.5 s per close on every validator before the gate opened; durable vote records shared the ledger’s LMDB writer, so the voter waited behind block processing; and the gate was opened only after the committee bookkeeping. Sixth, late notarization first shared a vote generator with the consensus instantiation’s notarization votes, which then queued behind thousands of late votes and slowed the close, and each late vote opened an instance of the closing epoch at its receivers, thousands of which were erased at installation.

**Limitations.** Each variant ran once, so the table gives no run-to-run variance; in earlier repeated runs of this prototype, p95 varied by up to 40% between identical runs, and the matrix was run seven times during the day, once stopped after three variants, with one variant failing to finish in two of them before the defects above were repaired. All validators share one host, so CPU contention rather than network latency sets the delays, and the fixed offered load measures neither saturation capacity nor wide-area behavior. The two arms were not interleaved in one pass, and the baseline rows predate the RAI rows by a day. The departures listed above mean these results validate neither the consensus instantiation’s durability nor a manifest covering admission witnesses.

## 10 Conclusion

RAI makes epoch transitions for account-local payments pipelined and safe. The next epoch’s voting starts at each validator’s local boundary while the previous epoch closes; immutable reports expose enough of the outgoing committee’s released votes to preserve latent and late finality; a deterministic construction over eligible report selections produces the checkpoint; and a predecessor gate with a narrow

prefix-wide overlap exception, fed by the outgoing committee’s late notarizations, keeps successor finality compatible with whatever the outgoing committee could still certify. The protocol defines the reports and the eligibility of checkpoint candidates and leaves reconstruction and agreement to two abstract services, convergence and consensus, with stated interfaces; safety and termination follow from their safety and termination together with explicit fault, data, and progress assumptions. A prototype instantiating them with rateless set reconciliation and a Kudzu-style election settles all nine workload variants, including forks with one offline or one Byzantine validator, keeping non-fork median latency near 100 ms across three checkpoints; late notarization lets blocks published during a close finalize before the checkpoint, so that without forks the tails stay below those of the unmodified node, while under forks the blocks whose late notarization splits still wait for the checkpoint.

**Use of AI tools.** ChatGPT assisted with an earlier restructuring of the manuscript, source checking, protocol review, and proof exposition. Claude Code assisted with the prototype implementation, the benchmark harness, running the experiments, diagnosing the defects, and drafting this revision, including the abstraction of the two services. The proof arguments have not been machine checked and require author review.

## References

- [1] Mathieu Baudet, George Danezis, and Alberto Sonnino. 2020. FastPay: High-Performance Byzantine Fault Tolerant Settlement. In *Proceedings of the ACM Conference on Advances in Financial Technologies (AFT)*. doi:10.1145/3419614.3423249
- [2] Mathieu Baudet, Alberto Sonnino, Mahimna Kelkar, and George Danezis. 2023. Zef: Low-latency, Scalable, Private Payments. In *Proceedings of the Workshop on Privacy in the Electronic Society (WPES)*. 1–16. doi:10.1145/3603216.3624952
- [3] Sam Blackshear, Andrey Chursin, George Danezis, Anastasios Kichidis, Leferis Kokoris-Kogias, Xun Li, Mark Logan, Ashok Menon, Todd Nowacki, Alberto Sonnino, Brandon Williams, and Lu Zhang. 2024. Sui Lutris: A Blockchain Combining Broadcast and Consensus. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2606–2620. doi:10.1145/3658644.3670286
- [4] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. 2001. Secure and Efficient Asynchronous Broadcast Protocols. In *Advances in Cryptology – CRYPTO 2001 (LNCS, Vol. 2139)*. Springer, 524–541. doi:10.1007/3-540-44647-8\_31
- [5] Daniel Collins, Rachid Guerraoui, Jovan Komatovic, Matteo Monti, Athanasios Xygkis, Matej Pavlovic, Petr Kuznetsov, Yvonne-Anne Pignolet, Dragos-Adrian Seredinschi, and Andrei Tonkikh. 2020. Online Payments by Merely Broadcasting Messages (Extended Version). doi:10.48550/arXiv.2004.13184 arXiv:2004.13184 Extended version of the DSN 2020 paper; Appendix A describes asynchronous reconfiguration.
- [6] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the Presence of Partial Synchrony. *J. ACM* 35, 2 (1988), 288–323. doi:10.1145/42282.42283
- [7] David Eppstein, Michael T. Goodrich, Frank Uyeda, and George Varghese. 2011. What’s the Difference? Efficient Set Reconciliation without Prior Context. In *Proceedings of the ACM SIGCOMM 2011 Conference*. 218–229. doi:10.1145/2018436.2018462
- [8] Michael T. Goodrich and Michael Mitzenmacher. 2011. Invertible Bloom Lookup Tables. In *49th Annual Allerton Conference on Communication, Control, and Computing*. 792–799. doi:10.1109/Allerton.2011.6120248
- [9] Rachid Guerraoui, Jovan Komatovic, Petr Kuznetsov, Yvonne-Anne Pignolet, Dragos-Adrian Seredinschi, and Andrei Tonkikh. 2020. Dynamic Byzantine Reliable Broadcast. doi:10.48550/arXiv.2001.06271 arXiv:2001.06271 Extended technical report, version 2.
- [10] Rachid Guerraoui, Petr Kuznetsov, Matteo Monti, Matej Pavlovic, and Dragos-Adrian Seredinschi. 2019. The Consensus Number of a Cryptocurrency. In *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 307–316. doi:10.1145/3293611.3331589
- [11] Leferis Kokoris-Kogias, Alberto Sonnino, and George Danezis. 2023. Cuttlefish: Expressive Fast Path Blockchains with FastUnlock. doi:10.48550/arXiv.2309.12715 arXiv:2309.12715 Version 1.
- [12] Petr Kuznetsov, Yvonne-Anne Pignolet, Pavel Ponomarev, and Andrei Tonkikh. 2021. Permissionless and Asynchronous Asset Transfer. In *35th International Symposium on Distributed Computing (DISC 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 209)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 28:1–28:19. doi:10.4230/LIPIcs.DISC.2021.28
- [13] Petr Kuznetsov and Andrei Tonkikh. 2022. Asynchronous reconfiguration with Byzantine failures. *Distributed Computing* 35 (2022), 477–502. doi:10.1007/s00446-022-00421-1
- [14] Colin LeMahieu. [n. d.]. Nano: A Feeless Distributed Cryptocurrency Network. White paper. [https://content.nano.org/whitepaper/Nano\\_Whitepaper\\_en.pdf](https://content.nano.org/whitepaper/Nano_Whitepaper_en.pdf)
- [15] RsNano contributors. [n. d.]. RsNano software repository. Software repository. <https://github.com/rsnano-node/rsnano-node> Base of the RAI prototype and of the evaluated baseline.
- [16] Linggang Shangguan. 2026. Transaction-Driven Dynamic Reconfiguration for Certificate-Based Payment Systems. doi:10.48550/arXiv.2601.09146 arXiv:2601.09146 Initial preprint, version 1, January 14, 2026.
- [17] Victor Shoup. 2024. Sing a Song of Simplex. In *38th International Symposium on Distributed Computing (DISC 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 319)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 37:1–37:22. doi:10.4230/LIPIcs.DISC.2024.37
- [18] Victor Shoup, Jakub Sliwinski, and Yann Vonlanthen. 2025. Kudzu: Fast and Simple High-Throughput BFT. In *39th International Symposium on Distributed Computing (DISC 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 356)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 42:1–42:19. doi:10.4230/LIPIcs.DISC.2025.42
- [19] Lei Yang, Yossi Gilad, and Mohammad Alizadeh. 2024. Practical Rateless Set Reconciliation. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 595–612. doi:10.1145/3651890.3672219